

第17讲实验设计：Softmax、Temperature与数值稳定性

实验目标不是“跑通代码”，而是通过对照实验验证 Softmax 的关键性质。

核心研究式问题： Softmax
 如何把相对分数变成概率？哪些变换改变结果，哪些不改变？实现细节什么时候会造成 inf/nan？

实验环境

项目	建议
Python	3.10+
NumPy	必需
Matplotlib	用于结果图
PyTorch	可选，本材料环境已用于对照
GPU	不需要，CPU 足够

实验原则

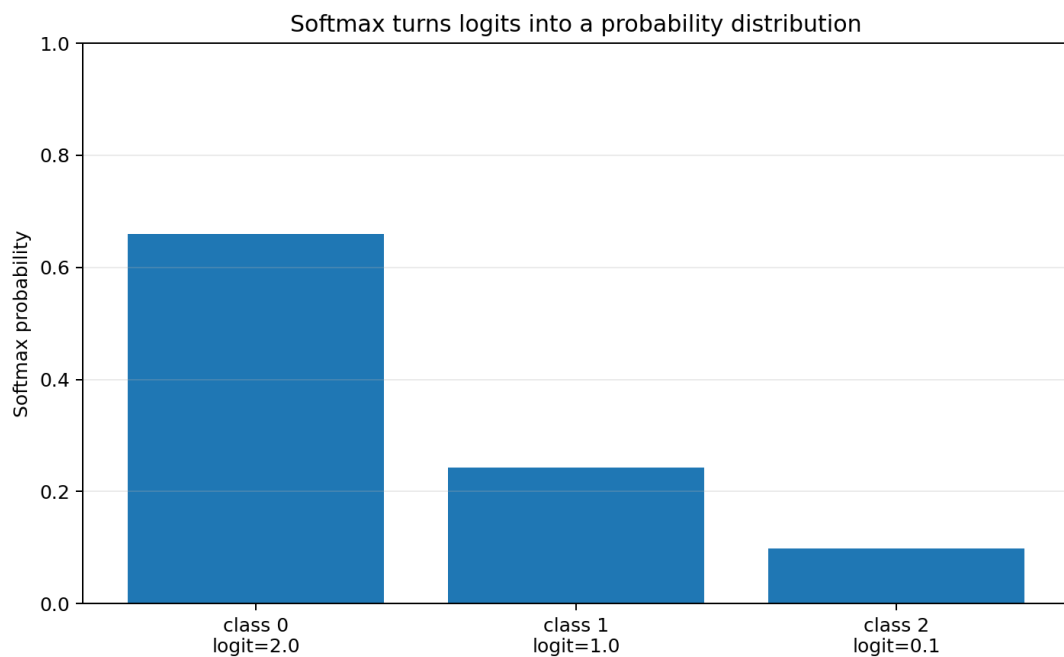
- 每个实验先写“预测”，再运行代码。
- 同时记录输入、Shape、输出和关键统计量。
- 出现 inf/nan 不直接改代码，先解释产生原因。
- PyTorch 只做验证；核心 Softmax 先用 NumPy 自己写。

实验1: 手工 Softmax

输入 $\text{logits}=[2.0, 1.0, 0.1]$ 。分别输出 $\exp(\text{logits})$ 、 $\exp$ 总和、最终 probabilities。

变量	控制方式
自变量	固定 logits
因变量	$\exp$ 、概率
验证标准	概率和=1; argmax 前后相同

实际结果: $\text{probabilities}=[0.659001, 0.242433, 0.098566]$, 总和=1.0。



实验2：整体平移不变性

分别计算 logits 、 $\text{logits}+10$ 、 $\text{logits}-7.5$ 、 $\text{logits}+100$ 的 Softmax。

假设：结果应完全相同，因为公共 $\exp(c)$ 在分子/分母中抵消。

实际：三组 max_diff 均为 0.000000000000。

科研意义：这是“稳定版 Softmax 可以先减 max ”的理论依据，不是经验 hack。

实验3：数值稳定性

输入 $[1000, 1001, 999]$ 。对比 naive exp 与 stable softmax。

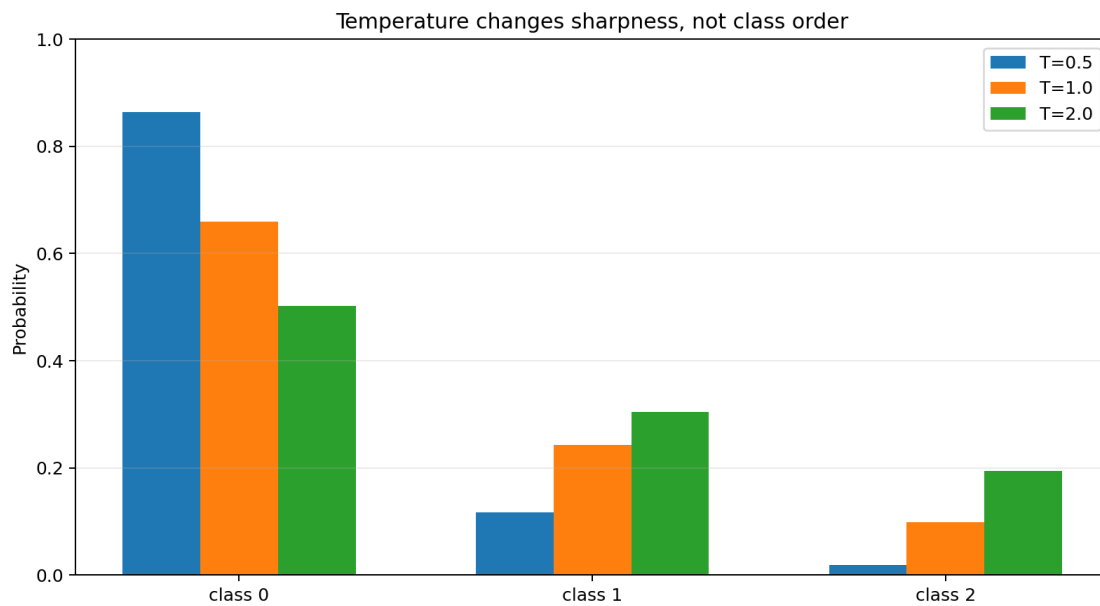
实现	中间结果	最终结果
naive	$[\text{inf}, \text{inf}, \text{inf}]$	$[\text{nan}, \text{nan}, \text{nan}]$
stable: 先减 max	$[-1, 0, -2]$	$[0.244728, 0.665241, 0.090031]$

实验4: Temperature

固定 $\text{logits}=[2,1,0.1]$ ，只改变 T 。

T	最大概率	Entropy (仅比较用)
0.5	0.863777	0.453668
1.0	0.659001	0.846738
2.0	0.501688	1.026244

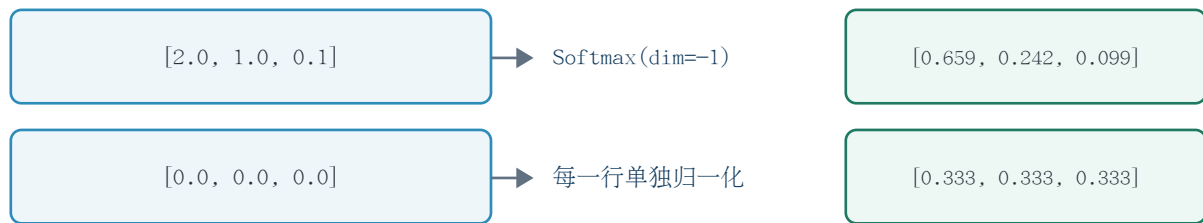
观察: T 变大时最大概率下降、熵升高, 分布更平。



实验5: Batch / dim

输入 `shape=(2,3)`，沿最后一维 `Softmax`。验证输出 `Shape` 不变，并检查每行 `sum`。

实际输出: `row sums=[1.0,1.0]`。第二行 `logits` 全为 0，因此得到均匀分布 `[1/3,1/3,1/3]`。



实验6: 微型 LLM vocabulary

token	logit	probability
猫	1.20	0.193357
狗	2.10	0.475581
AI	-0.30	0.043144
你好	0.80	0.129611
世界	1.00	0.158307

最大概率 `token=狗`，但这只表示在该微型分布中它最受偏好；不代表语言事实“狗一定正确”。

实验7: NumPy vs PyTorch

NumPy 稳定版与 `torch.softmax` 对比: 最大绝对差异=0.0。

这一步的意义不是证明 PyTorch “正确”, 而是建立 API 与你已经理解的底层数学之间的对应关系。

实验代码中的关键函数

```
def softmax_stable(logits):  
    logits = np.asarray(logits, dtype=np.float64)  
    m = np.max(logits, axis=-1, keepdims=True)  
    shifted = logits - m  
    exp_values = np.exp(shifted)  
    return exp_values / np.sum(exp_values, axis=-1, keepdims=True)
```

实验结果如何记录

字段	本讲建议记录
Question	Softmax 的关键性质是否与数学推导一致?
Hypothesis	平移不变; T 改变尖锐度; 稳定版避免 overflow
Independent variables	常数 c、Temperature T、实现方式
Dependent variables	probability、sum、max_prob、是否出现 inf/nan
Controls	固定 logits, 除被研究因素外其余不变
Result	逐实验保存实际数值和图
Interpretation	解释原因, 而不是只说 “符合预期”

最小科研闭环: 现象 -> 假设 -> 控制变量 -> 实验 -> 数值结果 -> 解释。即使只是 Softmax 基础实验, 也按照这个结构练习; 以后做 Agent Systems 机制实验时结构完全相通。

实验验收与常见错误

- ☐ 能自己写 stable softmax, 不复制答案。
- ☐ 能故意复现 [1000,1001,999] 的 naive overflow, 并解释原因。
- ☐ 能预测 T 改变后的分布方向。
- ☐ 能把 Batch Softmax 的 dim 和 Shape 说清楚。
- ☐ 能解释 NumPy 与 PyTorch 为什么结果相同。

常见错误:

忘记 axis / dim, 导致把不该竞争的维度混在一起。

直接 exp 大 logits, 得到 inf/nan。

把 temperature 当模型参数更新。

把 Softmax probability 当已经校准的真实置信度。

下一讲实验接口: Cross Entropy 会直接使用本讲的“正确类别概率”, 并通过 $-\log$ 把它变成 Loss; 随后才能进入 Gradient。